```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from skimage import data
from sklearn.cluster import KMeans
```

```
gray_img = cv2.imread('/content/e-6.jpg', cv2.IMREAD_GRAYSCALE)
color_img = cv2.cvtColor(cv2.imread('/content/e-6.jpg'), cv2.COLOR_BGR2RGB)
```

```
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1); plt.imshow(gray_img, cmap='gray'); plt.title('Grayscale Image');
plt.subplot(1, 2, 2); plt.imshow(color_img); plt.title('Color Image'); plt.axis('off')
plt.tight_layout()
plt.show()
```




```
blur = cv2.GaussianBlur(gray_img, (5, 5), 0)

plt.figure(figsize=(8, 4))
plt.subplot(1, 2, 1); plt.imshow(gray_img, cmap='gray'); plt.title('Original'); plt.ax
plt.subplot(1, 2, 2); plt.imshow(blur, cmap='gray'); plt.title('After Gaussian Blur');
plt.tight_layout()
plt.show()
```




```
_, global_thresh = cv2.threshold(blur, 127, 255, cv2.THRESH_BINARY)

plt.figure(figsize=(5, 5))
plt.imshow(global_thresh, cmap='gray')
plt.title('Global Thresholding (T=127)')
```

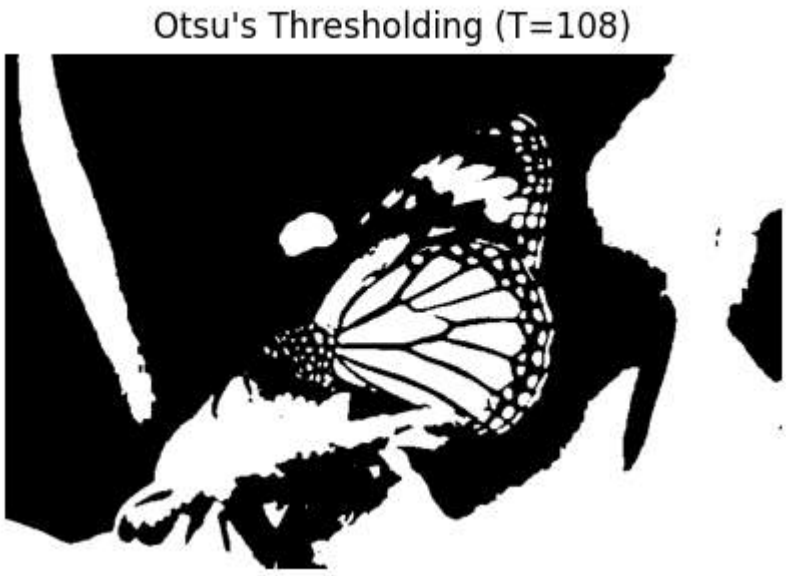
```
plt.axis('off')
plt.show()
```




```
otsu_val, otsu_thresh = cv2.threshold(blur, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTS
print(f"Otsu's optimal threshold value: {otsu_val}")

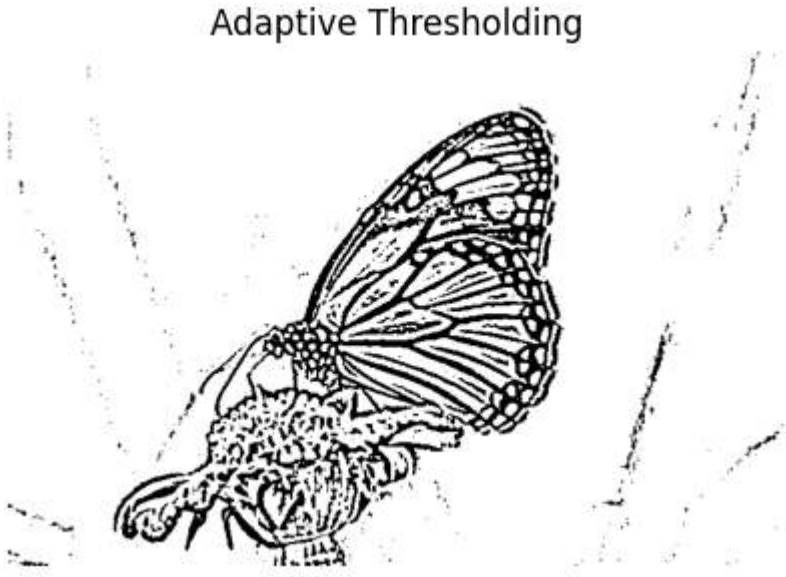
plt.figure(figsize=(5, 5))
plt.imshow(otsu_thresh, cmap='gray')
plt.title(f"Otsu's Thresholding (T={otsu_val:.0f})")
plt.axis('off')
plt.show()
```

```
Otsu's optimal threshold value: 108.0
```




```
adaptive_thresh = cv2.adaptiveThreshold(
    blur, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY, 11, 2
)

plt.figure(figsize=(5, 5))
plt.imshow(adaptive_thresh, cmap='gray')
plt.title('Adaptive Thresholding')
plt.axis('off')
plt.show()
```




```
_, thresh_ws = cv2.threshold(blur, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)

kernel = np.ones((3, 3), np.uint8)
opening = cv2.morphologyEx(thresh_ws, cv2.MORPH_OPEN, kernel, iterations=2)
sure_bg = cv2.dilate(opening, kernel, iterations=3)

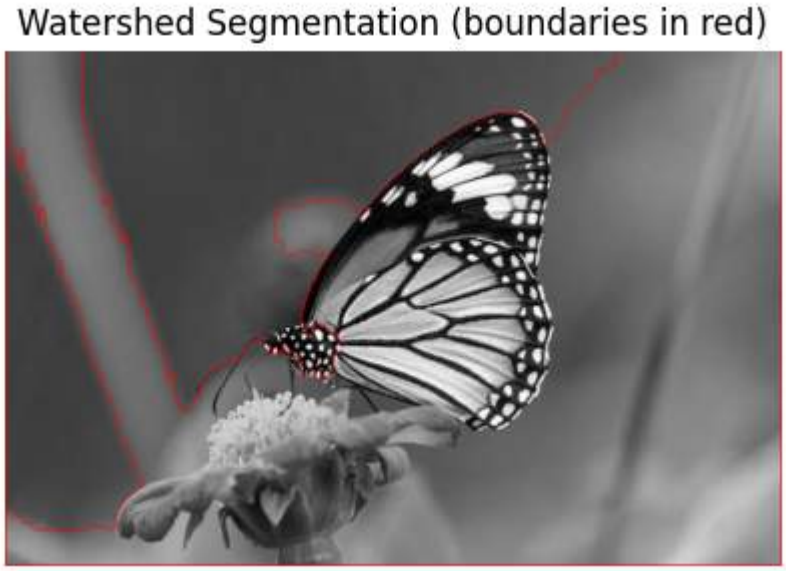
dist_transform = cv2.distanceTransform(opening, cv2.DIST_L2, 5)
_, sure_fg = cv2.threshold(dist_transform, 0.5 * dist_transform.max(), 255, 0)
sure_fg = np.uint8(sure_fg)

unknown = cv2.subtract(sure_bg, sure_fg)
_, markers = cv2.connectedComponents(sure_fg)
markers = markers + 1
markers[unknown == 255] = 0

color_img_for_ws = cv2.cvtColor(gray_img, cv2.COLOR_GRAY2BGR)
markers = cv2.watershed(color_img_for_ws, markers)

watershed_result = color_img_for_ws.copy()
watershed_result[markers == -1] = [0, 0, 255]  # mark boundaries in red

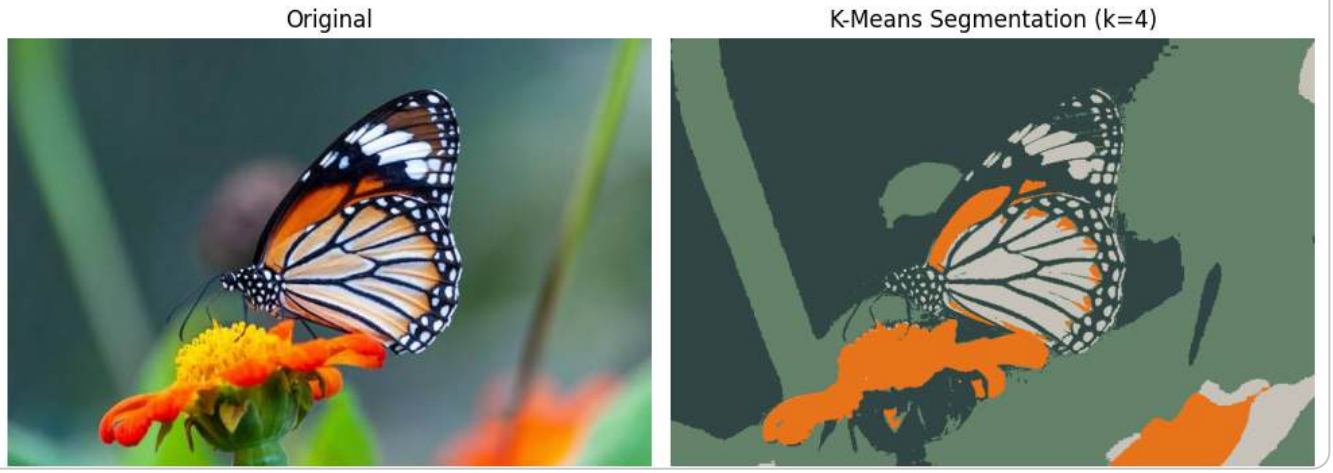
plt.figure(figsize=(5, 5))
plt.imshow(cv2.cvtColor(watershed_result, cv2.COLOR_BGR2RGB))
plt.title('Watershed Segmentation (boundaries in red)')
plt.axis('off')
plt.show()
```



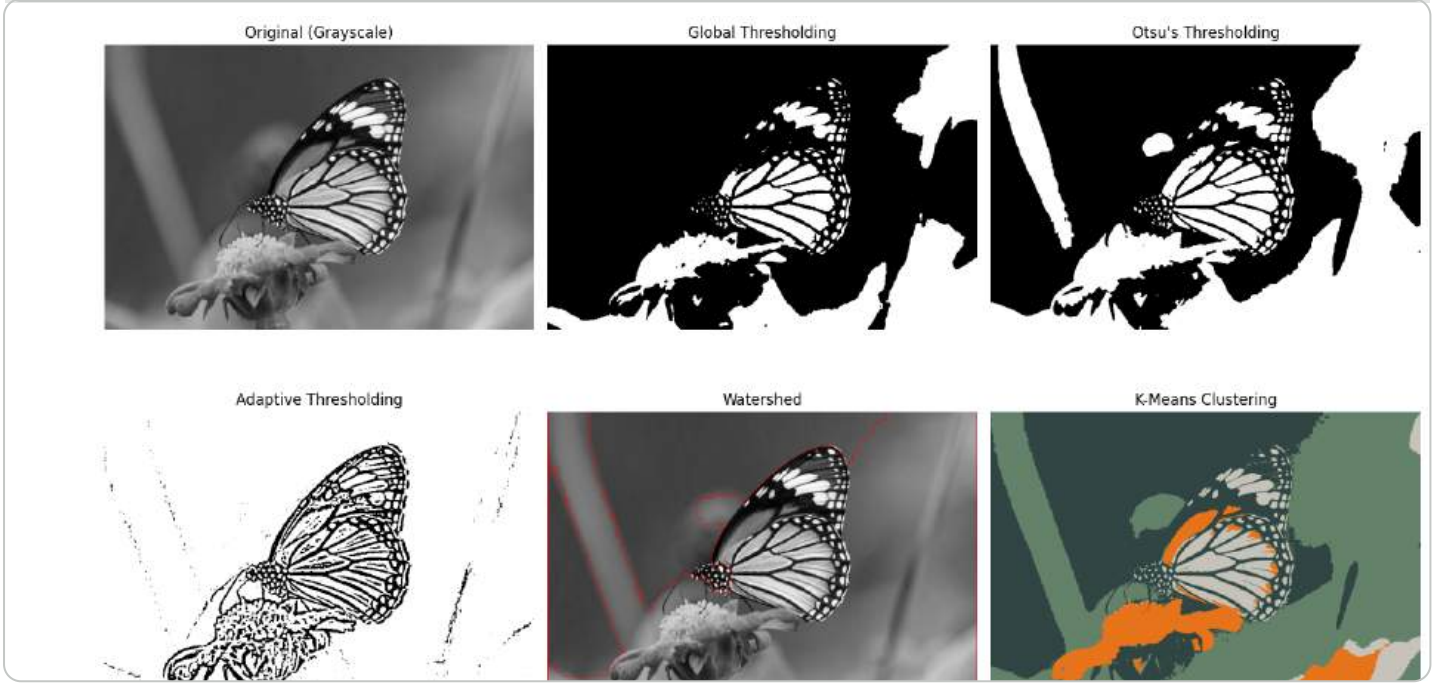

```
pixels = color_img.reshape((-1, 3)).astype(np.float32)

k = 4  # number of color clusters
kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
labels = kmeans.fit_predict(pixels)
centers = np.uint8(kmeans.cluster_centers_)
segmented_kmeans = centers[labels].reshape(color_img.shape)

plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1); plt.imshow(color_img); plt.title('Original'); plt.axis('off')
plt.subplot(1, 2, 2); plt.imshow(segmented_kmeans); plt.title(f'K-Means Segmentation (
plt.tight_layout()
plt.show()
```




```
fig, axes = plt.subplots(2, 3, figsize=(15, 9))
axes[0,0].imshow(gray_img, cmap='gray'); axes[0,0].set_title('Original (Graysc
axes[0,1].imshow(global_thresh, cmap='gray'); axes[0,1].set_title('Global Thre
axes[0,2].imshow(otsu_thresh, cmap='gray'); axes[0,2].set_title("Otsu's Thresh
axes[1,0].imshow(adaptive_thresh, cmap='gray'); axes[1,0].set_title('Adaptive
axes[1,1].imshow(cv2.cvtColor(watershed_result, cv2.COLOR_BGR2RGB)); axes[1,1]
axes[1,2].imshow(segmented_kmeans); axes[1,2].set_title('K-Means Clustering');
plt.tight_layout()
plt.show()
```


Original (Grayscale)
Global Thresholding
Otsu's Thresholding
Adaptive Thresholding
Watershed
K-Means Clustering